Sprint 3 · Del dato al modelo en la nube

¿podemos tener un primer modelo funcionando, sin programar?

FECHAS
23–27/10

HORAS
5 h de teoría (T) · 5 h de práctica (P) · 4 h de proyecto (PBL) = 14 h

Qué aprenderás

1. A **consultar y modificar** una base de datos relacional con SQL (`SELECT`, `JOIN`, `GROUP BY`, subconsultas, `INSERT/UPDATE/DELETE`).
2. A **modelar** datos con claves, restricciones e índices, y a explicar por qué importan.
3. A guardar el mismo dato como **documentos** en MongoDB y a decidir cuándo conviene SQL o NoSQL.
4. A lanzar un **experimento de AutoML**, leer sus métricas y elegir un umbral con criterio de negocio.
5. A **servir un modelo por API** y a controlar y cerrar los recursos que usas.

Requisitos previos

- Haber terminado el Sprint 2 (cuentas y herramientas preparadas).
- Un navegador actual. Para SQL: **DB Browser for SQLite** instalado, o un cliente SQLite en línea (consulta con tu docente cuál).
- No hace falta saber programar en este sprint. Los ejercicios de SQL y AutoML no usan Python; solo el laboratorio opcional de NoSQL (y el laboratorio de AutoML basado en cuadernos, si se usa).
- Datos: `turisdata.db` (SQLite con las tablas `alojamientos`, `clientes` y `reservas`). Son datos **sintéticos**: sirven para practicar, no son estadísticas reales del turismo en Canarias.

Prácticas

Bloque A · Bases de datos (sin Python)

Abre `turisdata.db` en DB Browser for SQLite, pestaña **Ejecutar SQL**, y sigue los scripts en orden. En cada uno hay ideas con ejemplos y ejercicios; la solución la comprobarás con tu docente o comparando tus resultados con la descripción del ejercicio. Los ejercicios marcados "reto" son opcionales.

Script	Tema	Tiempo
`sql/01_select_where.sql`	`SELECT`, `WHERE`, `ORDER BY`, `LIMIT`, `NULL`	20 min
`sql/02_join.sql`	`JOIN` de dos y tres tablas, `LEFT JOIN`	20 min
`sql/03_group_by.sql`	Agregaciones, `GROUP BY`, `HAVING`	25 min
`sql/04_subconsultas.sql`	Subconsultas y `WITH` (CTE)	20 min
`sql/05_crud.sql`	`INSERT`, `UPDATE`, `DELETE` y transacciones (sobre una copia)	20 min
`sql/06_modelado_indices.sql`	Claves, restricciones, normalización e índices	25 min

Guía / cuaderno	Tema	Tiempo
`NoSQL_MongoDB.md` + `S03_01_nosql_mongomock` (opcional)	El mismo dato de reservas como documentos; consultas y agregaciones con MongoDB	60 min

Bloque B · AutoML y modelo en producción

La guía de AutoML se publica el 23/10, una vez confirmada la plataforma. El flujo es siempre el mismo: dato, experimento, evaluación, explicabilidad, servicio por API e informe de recursos. Tu docente te dirá qué ruta seguimos; el reto y el entregable son los mismos.

RETO Reto PBL · «Primer modelo en producción» (4 h)

En pocas palabras (léelo primero)

Qué os pide el cliente. TurisData ha perdido dinero porque algunas reservas se cancelan a última hora y la habitación se queda vacía. Os envía su histórico de reservas y os pide: *«¿Se puede construir algo que avise de qué reservas tienen más riesgo de cancelarse, y que mi equipo de reservas lo consulte desde otra aplicación?»* Es decir: **un modelo de IA funcionando como un servicio**, no solo un experimento.

Este es vuestro primer modelo. No hace falta saber programar: usaréis una plataforma de **AutoML** (una herramienta que prueba varios modelos por vosotros y os muestra los resultados). Vuestro trabajo es **entender lo que hace**, **elegir bien** y **explicárselo al cliente**.

Qué vais a hacer, en seis pasos:

1. **Mirar los datos** con SQL (las consultas que habéis practicado) y decidir qué se quiere predecir.
2. **Quitar las columnas «trampa»** que no se conocen en el momento de reservar (eso se llama *fuga de datos*).
3. **Lanzar un experimento** de AutoML y comparar los modelos que salgan.
4. **Elegir un umbral** (cuándo decimos «esta reserva tiene riesgo alto») pensando en dinero, no en números sueltos.
5. **Publicar el modelo como servicio (API)**, hacer una llamada real, y **cerrar y borrar** todo lo que hayáis creado.
6. **Redactar el informe** (4 páginas máximo) con todo lo anterior.

Qué vais a entregar: un único documento de máximo 4 páginas con tres partes: informe del mejor modelo, evidencia de la llamada a la API e informe de recursos y costes.

Qué vais a aprender haciéndolo: el recorrido completo de un proyecto de IA, **del dato al servicio**, incluida una parte que mucha gente olvida: **vigilar el coste y cerrar los recursos**.

Cuánto tiempo: 4 horas de reto dentro del sprint (del 23 al 27/10).

Palabras que van a aparecer en este reto

Palabra	Qué significa, en sencillo
AutoML	Herramienta que entrena y compara varios modelos automáticamente
Variable objetivo	Lo que queremos predecir. Aquí: si la reserva se cancela (sí / no)
Variable (o columna)	Cada dato de una reserva: canal, tarifa, antelación, país…
Fuga de datos	Cuando el modelo usa información que **no se conocería a la hora de decidir**. Hace trampa y sus resultados parecen mejores de lo que son
Partición temporal	Entrenar con reservas **antiguas** y evaluar con reservas **más recientes**, como pasaría en la realidad
Métrica	Un número que dice lo bien o mal que lo hace el modelo (por ejemplo, la exactitud o el AUC)
Umbral	El punto de corte. Si el modelo dice «70 % de riesgo» y el umbral es 60 %, se da la alerta
Falsa alarma	El modelo avisa de cancelación y la reserva se mantiene
Cancelación no detectada	El modelo no avisa y la reserva se cancela
API	Una puerta por la que otra aplicación puede preguntar al modelo y recibir una respuesta
Recurso (en la nube)	Todo lo que creáis en la plataforma: datos subidos, experimentos, servicios activos. **Consumen cuota o dinero**
Explicabilidad	Saber **qué variables** pesan más en la decisión del modelo

Contexto del cliente

TurisData Canarias nos ha enviado el histórico de sus reservas. Cada cancelación de última hora le deja una habitación vacía y le cuesta ingresos. Quiere saber si un modelo puede **avisar de las reservas con más riesgo de cancelarse** y que su equipo de reservas lo consulte desde otra aplicación. Vosotros sois el equipo que lo pone en marcha, de los datos al servicio.

Qué tenéis que entregar

Un único documento (Markdown o PDF, máximo 4 páginas) con tres partes:

1. **Informe del mejor modelo:** cómo se ha planteado el problema, qué modelos se compararon, métrica elegida y por qué, resultados en datos no vistos, umbral de decisión elegido con su justificación de negocio y variables más influyentes.

2. **Evidencia de la llamada a la API:** captura o texto de una petición real al servicio del modelo, con la respuesta (probabilidad de cancelación y alerta). Sin claves ni credenciales visibles.
3. **Informe de recursos y costes:** qué recursos se han creado, cuánto tiempo y capacidad se han usado, cuánto ha costado o cuánto habría costado, y **prueba de que se han cerrado o eliminado** todos.

Restricciones

- **Tiempo:** 4 h de proyecto dentro del sprint.
- **Datos:** solo los datos sintéticos de TurisData. No subáis datos personales reales.
- **Recursos:** se trabaja con el límite de uso que indique el docente; todo recurso creado debe quedar cerrado o eliminado al terminar.
- **Seguridad:** nunca pongáis claves, contraseñas ni cadenas de conexión en el informe ni en repositorios.
- **Evitar la fuga de datos:** el modelo solo puede usar información disponible en el momento de hacer la reserva.

Criterios de evaluación (rúbrica del curso, 0–4 en cada criterio)

Criterio	Qué se mirará en este reto
Corrección técnica	Partición de datos adecuada, métrica coherente con el problema, comparación de modelos, umbral justificado, la API responde.
Reproducibilidad	Otra persona puede repetir el experimento con vuestro informe: datos, ajustes, versiones y pasos.
Análisis y comunicación al cliente	Conclusiones claras para alguien no técnico y recomendación accionable: qué hacer con una alerta y con qué límites.
IA responsable	Riesgos identificados (datos sintéticos, falsas alertas que perjudican a clientes, sesgos por país o canal) y medidas propuestas.

Pasos sugeridos

- **Día 1:** revisad el dato en la base (SQL), decidid la variable objetivo y las columnas que no se pueden usar (fuga). Preparad la plataforma elegida.
- **Día 2:** lanzad el experimento, comparad modelos, evaluad con una partición temporal y elegid el umbral. Publicad el servicio y haced la llamada.
- **Día 3:** cerrad y eliminad recursos, completad el informe de recursos, revisad con la rúbrica y preparad la demo.

Guía paso a paso para principiantes

Paso 1 · Entender el problema y los datos (día 1)

1. Releed la escena del cliente y respondeos con una frase: «*Queremos predecir* `__` ***para que el equipo de reservas pueda*** `____`».
2. Abrid `turisdata.db` en DB Browser for SQLite y mirad la tabla `reservas` con un `SELECT * ... LIMIT 20`. ¿Qué columnas hay?
3. Con `GROUP BY` calculad el **porcentaje de cancelaciones** por canal. Es una forma útil de empezar a conocer el dato.
4. **Decidid la variable objetivo** (la columna `cancelada`, sí/no).
5. **Quitad las columnas trampa.** Pregunta clave para cada columna: **«¿La conoceríamos en el momento de hacer la reserva?»** Si la respuesta es no (por ejemplo, un dato que solo existe después de cancelar), no puede entrar en el modelo. Anotad cuáles habéis excluido y por qué: **es parte del informe**.
6. Preparad la plataforma que indique el docente (leed la guía de AutoML de la ruta elegida: en esta carpeta tenéis `Guia_AutoML_AutoGluon.pdf` como ejemplo del flujo).

Paso 2 · Lanzar el experimento y comparar modelos (día 2, mañana)

1. **Partid los datos por fecha:** las reservas antiguas para entrenar, las más recientes para evaluar. (Si barajáis las filas al azar, el modelo «ve el futuro» y los resultados parecen mejores de lo que serían en la realidad.)
2. Lanzad el experimento con el límite de tiempo y de uso que indique el docente.
3. Cuando termine, mirad la **tabla de modelos**. No elijáis solo por el número más alto: apuntad qué métrica usáis y **por qué es la adecuada para este problema** (por ejemplo: si solo hay un 20 % de cancelaciones, un modelo que dice «nadie cancela» acierta el 80 % y no sirve para nada).
4. Mirad las **variables más influyentes** (explicabilidad). ¿Tienen sentido para el negocio?

Paso 3 · Elegir el umbral con criterio de negocio (día 2, tarde)

1. Pensad en las dos formas de equivocarse: - **Falsa alarma:** se actúa sobre una reserva que no se iba a cancelar (cuesta una llamada, o molestar a un cliente). - **Cancelación no detectada:** se pierde la oportunidad de revender la habitación (cuesta una noche vacía).

2. Estimad **cuál cuesta más** (aunque sea con una hipótesis: «una noche vacía son 100 €, una llamada son 5 €») y elegid un umbral coherente. Explicadlo con una frase que entienda un directivo.
3. Evaluad el modelo con el umbral elegido en los datos que **no ha visto**.

Paso 4 · Publicar el servicio y hacer una llamada (día 2–3)

1. Publicad el modelo siguiendo la guía (el modelo queda «escuchando» peticiones).
2. Haced **una llamada real**: enviad los datos de una reserva ficticia y recibid la **probabilidad de cancelación** y la alerta.
3. **Capturad la petición y la respuesta**, pero **sin claves ni contraseñas visibles**. Si aparece una clave en la captura, tapadla.

Paso 5 · Cerrar y borrar (día 3, importante)

1. **Parad y eliminad** el servicio publicado, el experimento y los datos subidos. Lo que se deja encendido **gasta cuota (o dinero)**.
2. Anotad en el informe: qué recursos creasteis, cuánto tiempo y capacidad usaron, cuánto ha costado o **habría costado** y **la prueba de que se han cerrado** (captura de la lista vacía de recursos).
3. Si algo no se puede borrar, **avisad al docente**.

Paso 6 · Redactar el informe (4 páginas máximo)

Estructura recomendada:

1. **El problema y la métrica** (media página).
2. **Qué modelos se compararon y cuál ganó**, con el umbral y su razón de negocio.
3. **Variables que pesan más** y **variables excluidas por fuga**.
4. **Evidencia de la llamada a la API.**
5. **Informe de recursos y costes**, con la prueba de cierre.
6. **Riesgos de IA responsable** (al menos dos) y **una medida** para cada uno.
7. **Recomendación al cliente:** qué hacer con una alerta y con qué límites.

Cómo usar la IA en este reto

Para qué	Ejemplo de petición	Qué tenéis que comprobar
Entender el SQL	«Explícame esta consulta SQL línea por línea: *[pega tu consulta]*»	Ejecutadla y comprobad que hace lo que dice
Escribir una consulta	«Dada una tabla `reservas(id, canal, cancelada)`, escribe una consulta que calcule el % de cancelaciones por canal»	Que los nombres de columnas sean los **vuestros** y que el resultado tenga sentido
Interpretar una métrica	«Explícame qué es el AUC con un ejemplo de hotel, sin fórmulas»	Que podáis explicarlo vosotros
Redactar para un directivo	«Reescribe este párrafo para alguien que no sabe estadística: *[párrafo]*. No cambies las cifras»	Que las cifras sean idénticas

Muy importante: nunca peguéis claves de la plataforma, contraseñas ni cadenas de conexión en una IA ni en el informe ni en el repositorio. Si por error ocurre, avisad al docente y **cambiad la clave**.

No uséis la IA para inventar resultados: todo número del informe debe salir de vuestro experimento.

Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
Resultados «demasiado buenos» (casi 100 %)	Fuga de datos: una columna delata la respuesta	Revisad cada columna: ¿se conocería al reservar?
Elegir el modelo solo por la métrica más alta	Es lo más visible	Explicad **por qué esa métrica** y qué significa en euros
Barajar las filas al azar antes de partir	Es lo que se hace «de normal»	Aquí hay tiempo: **entrenad con lo antiguo, evaluad con lo reciente**
Dejar el servicio encendido	Se olvida	Cread una tarjeta «Cerrar recursos» en el tablero y hacedla en el último rato
Capturas con claves visibles	Prisa	Revisad la captura antes de subirla
Informe que solo habla de técnica	Se escribe para el docente	Releed como si fuerais el director de TurisData

Un buen resultado se parece a...

- **Una frase de recomendación:** *«Recomendamos alertar cuando el riesgo supere el 55 %: así detectamos 7 de cada 10 cancelaciones y solo molestamos a 1 de cada 5 clientes que mantenían su reserva. Con una noche vacía valorada en 100 € y una llamada en 5 €, el sistema compensa.»* (Las cifras son un ejemplo.)
- **Variables excluidas:** «Excluimos `fecha_cancelacion` y `estado_final` porque solo existen después de cancelar».
- **Informe de recursos:** una tabla con *recurso · cuándo se creó · tiempo usado · coste · cerrado (sí/no) · prueba*.

Antes de entregar: autoevaluación rápida

Pregunta	Sí / No
¿El informe tiene las tres partes y cabe en 4 páginas?	
¿La métrica y el umbral están explicados en lenguaje de negocio?	
¿Hay evidencia de una llamada real a la API, sin credenciales?	
¿Explicamos qué variables hemos excluido y por qué?	
¿Hemos cerrado y eliminado todos los recursos y lo demostramos?	
¿Hay al menos dos riesgos de IA responsable y una medida?	
¿Otra persona ha revisado el informe con la rúbrica?	

Si os atascáis

- **El experimento falla o tarda mucho:** reducid el tamaño de los datos o el tiempo máximo, y avisad al docente.
- **No sabemos qué columna es una trampa:** haced una lista con todas las columnas y, a su lado, «¿se conoce al reservar? sí/no». Discutidla en equipo.
- **No encontramos cómo cerrar un recurso:** buscad en la plataforma «recursos» o «facturación»; si no lo veis, preguntad.
- Más ayuda: Cómo se trabaja un reto.

Qué se enseña en la review

Una demo de 5 minutos: (1) el problema y la métrica, (2) el mejor modelo y su umbral, (3) una llamada en directo a la API, (4) el informe de recursos con la prueba de cierre, (5) una recomendación para el cliente y un límite del modelo.

Definition of Done

- [ ] El informe tiene las tres partes y cabe en 4 páginas.
- [ ] La métrica y el umbral están justificados en lenguaje de negocio.
- [ ] Hay evidencia de una llamada real a la API, sin credenciales.
- [ ] Se explica qué variables se excluyeron por posible fuga de datos.
- [ ] Todos los recursos creados están cerrados o eliminados y hay prueba.
- [ ] Se mencionan al menos dos riesgos de IA responsable y una medida.
- [ ] Otra persona del grupo ha revisado el informe con la rúbrica.

Recursos

- Documentación de SQLite (lenguaje SQL): https://www.sqlite.org/lang.html
- DB Browser for SQLite: https://sqlitebrowser.org/
- Manual de MongoDB: https://www.mongodb.com/docs/manual/
- Documentación de PyMongo: https://pymongo.readthedocs.io/
- La guía de AutoML de la ruta elegida (ver arriba).